

Zaawansowane struktury danych

Kacper Rosner, Artur Kupiec

Zaawansowane struktury, czyli jakie?

Zaawansowane struktury danych wyróżniają się realizacją optymalizacji konkretnych zadań I/O i wyszukiwania / sortowania. Nie istnieje jednoznaczny podział na zaawansowane i podstawowe struktury danych, dlatego na potrzeby tej lekcji, zdefiniujemy zaawansowane struktury, jako takie, które są optymalne dla konkretnego problemu algorytmicznego.

Dlaczego używamy i tworzymy nowe struktury danych?

Struktury jako odpowiedź na problemy algorytmiczne

Struktury zaawansowane jako odpowiedź na ograniczenia czasowo - sprzętowe

- ograniczają zużywane miejsce
- ograniczają czas wyszukiwania, dodawania, sortowania danych
- są tworem stanowiącymi wariacje na temat podstawowych struktur

Mogą szczególnie zajmować się wyszukiwaniem w kontekście: - licznosci - sprawdzania obecności w zbiorze - wyszukiwania max/min

Mogą być przystosowane do szczególnego typu danych (int, string, graf, współrzędne)

Array

Array - zmienna, która jest w stanie posiadać więcej niż jedną wartość

- wszystkie dane mają ten sam typ
- jej wielkość jest określona i nie da się jej zmienić

(implementacja w języku java - język python nie posiada standardowej tablicy)

```
public class Array { new *
    public static void main(String[] args) { new *

        String[] student = new String[4]; // rozmiar 4
        student[0] = "krzysztof";
        student[1] = "wojciech";
        student[2] = "leonarda";
        student[3] = "jan";
        System.out.println(student[1]); // wojciech
        student[4] = "radek"; // błąd

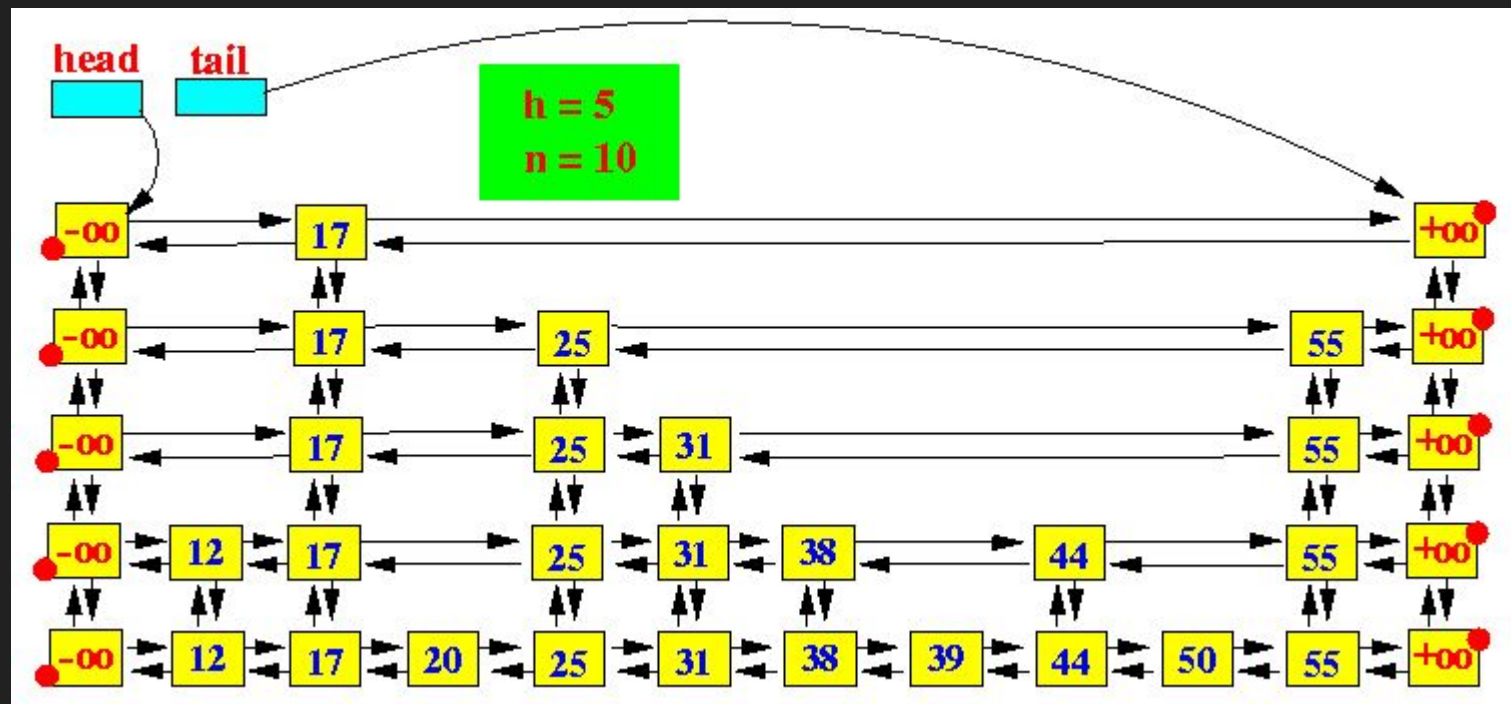
        String[] wydzial = {"WIEiT", "EAIIB", "FiIS"}; // rozmiar 3
        System.out.println(wydzial[0]); // WIEiT

        wydzial[2] = "IMiIP";
        System.out.println(wydzial[2]); // IMiIP
        wydzial[3] = "ILiGZ"; // błąd
    }
}
```

ArrayList

Operation	Time Complexity	Space Complexity
Inserting Element in ArrayList	$O(1)$	$O(N)$
Removing Element from ArrayList	$O(N)$	$O(1)$
Traversing Elements in ArrayList	$O(N)$	$O(N)$
Replacing Elements in ArrayList	$O(1)$	$O(1)$

Skip list - jak usprawnić wyszukiwanie?



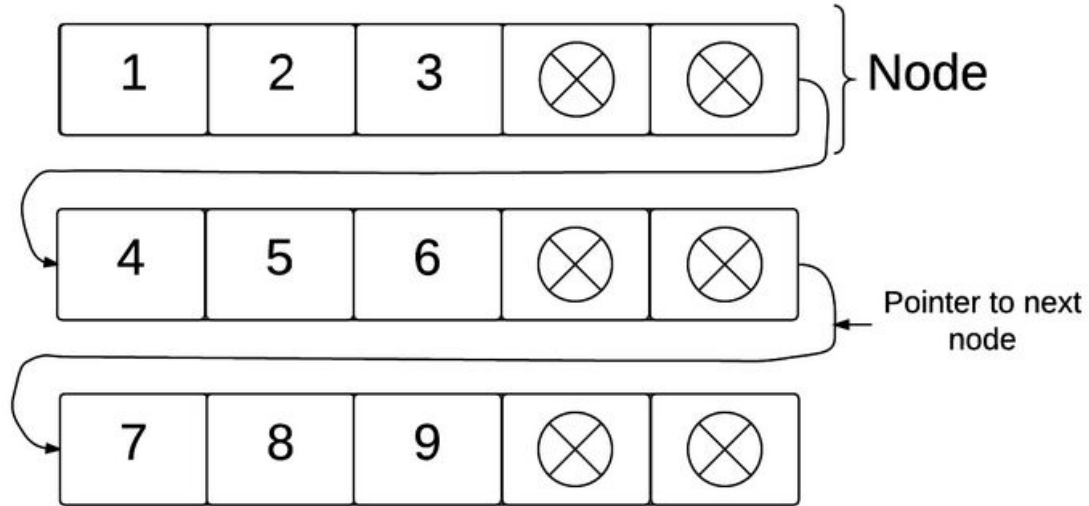
Linked list - jak usprawnić wstawianie/usuwanie?

Zbiór węzłów, w którym każdy z węzłów posiada 1 porcję danych oraz wskaźnik do następnego węzła

	Arrays	Linked Lists
<i>An existing data structure in the programming language</i>	Yes	No
<i>Fixed size in memory</i>	Yes	No
<i>Elements, or nodes, are stored right after each other in memory (contiguously)</i>	Yes	No
<i>Memory usage is low (each node only contains data, no links to other nodes)</i>	Yes	No
<i>Elements, or nodes, can be accessed directly (random access)</i>	Yes	No
<i>Elements, or nodes, can be inserted or deleted in constant time, no shifting operations in memory needed.</i>	No	Yes

Unrolled Linked List

Properties



Stack

- struktura LIFO
- pop, push, peek

Używamy, gdy interesuje nas historia operacji,
np. historia karty przeglądarki

```
stos = []

stos.append(1)
stos.append(2)
stos.append(3)
stos.append(4)
stos.append(5)

print(stos) # [1, 2, 3, 4, 5]
print(stos.pop()) # 5

stos_pop = stos.pop()
print(stos_pop) # 4
print(stos) # [1, 2, 3]

print(stos[-1]) # 3
print(stos) # [1, 2, 3]
```

Queue

- struktura FIFO
- enqueue, dequeue, peek

Używamy kiedy interesuje nas kolejność dodania nowych danych, np. kolejka drukowania,

```
kolejka = []
```

```
kolejka.append(1)
```

```
kolejka.append(2)
```

```
kolejka.append(3)
```

```
kolejka.append(4)
```

```
kolejka.append(5)
```

```
print(kolejka) # [1, 2, 3, 4, 5]
```

```
print(kolejka.pop(0)) # 1
```

```
kolejka_dequeue = kolejka.pop(0)
```

```
print(kolejka_dequeue) # 2
```

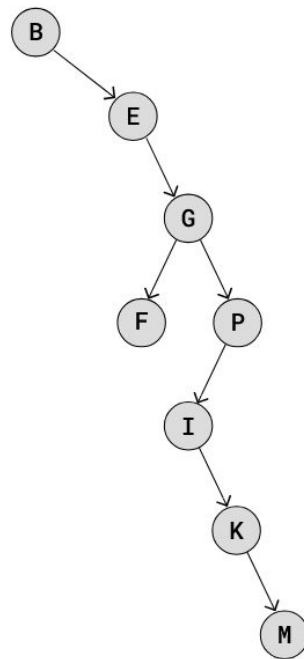
```
print(kolejka) # [3, 4, 5]
```

```
print(kolejka[0]) # 3
```

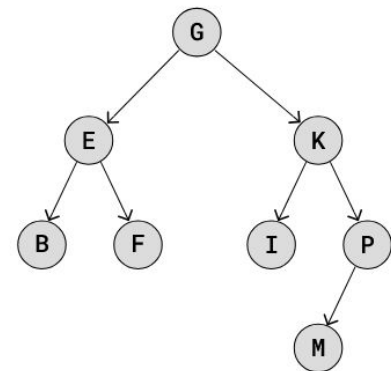
```
print(kolejka) # [3, 4, 5]
```

Tree

- struktura hierarchiczna
- korzeń, węzły, liście, krawędzie
- drzewa binarne
- binarne drzewa przeszukiwań
- drzewa czerwono-czarne/drzewa AVL



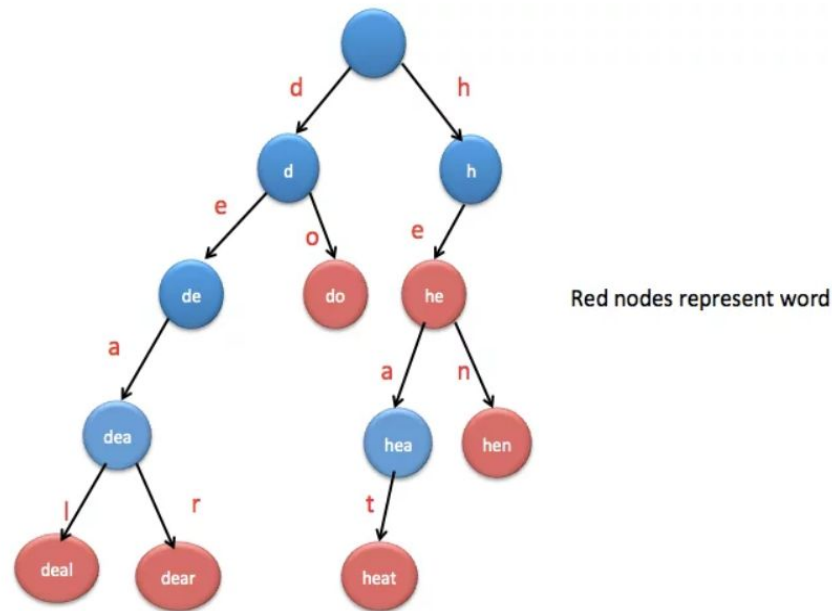
*Binary Search Tree
(unbalanced)
Height: 6*



*AVL Tree
(self-balancing)
Height: 3*

Trie

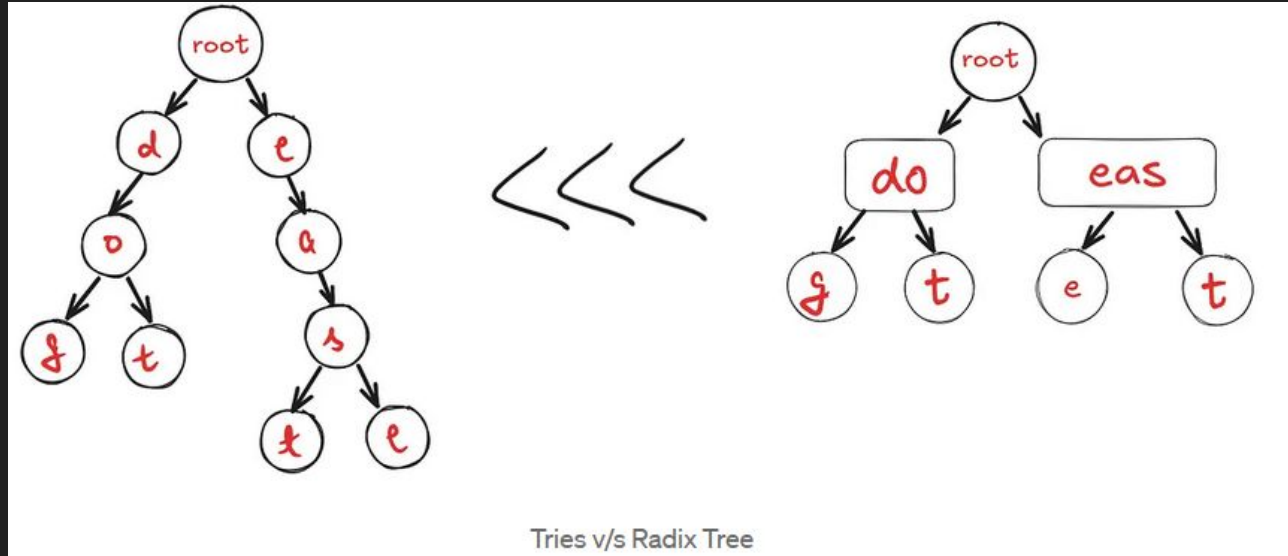
- drzewo przeszukiwań przechowujące w węzłach fragmenty kluczy
- przyspiesza wyszukiwanie, gdy koszt porównania całego klucza jest duży
- wymaga dużo pamięci
- insert, search, is_prefix
- autouzupełnianie, wyszukiwarki



Trie Data structure

Radix Tree (Compressed Trie)

- Trie, ale zamiast pojedynczych liter łączymy wspólne fragmenty
- wykorzystywane w routingu IP



Bloom Filter - probabilistyczna struktura danych

- filtr Blooma to probabilistyczna struktura danych
- służy do szybkiego sprawdzania przynależności elementu do zbioru
- może generować fałszywe pozytywy, ale nigdy fałszywe negatywy
- oszczędza miejsce kosztem pewnej niepewności co do wyniku

Cuckoo Hashing

1. Obliczamy 2 możliwe miejsca dla elementu (np. $h_1(x)$ i $h_2(x)$)
2. Jeśli jedno miejsce jest wolne => wstawiamy element
3. Jeśli zajęte => wyrzucamy istniejący element
4. Wyrzucony element trafia do swojej alternatywnej pozycji
5. Proces może się powtarzać (łańcuch przesunięć)

Cuckoo Hashing rozwiązuje problem kolizji w tablicach haszujących, zapewniając szybkie i przewidywalne wyszukiwanie bez tworzenia długich łańcuchów danych.

Trwałe struktury danych

- nie nadpisują danych
- tworzą nowe wersje
- stare wersje nadal istnieją
- używają współdzielenia pamięci
- GIT, snapshot VM, ctrl + z

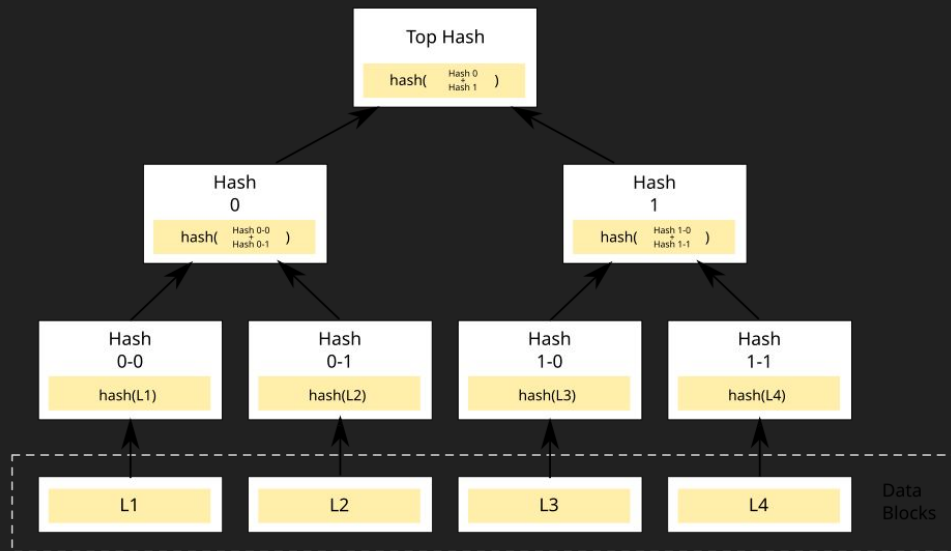


Dlaczego to działa?

Struktury trwałe zajmują mniej miejsca, ponieważ nie kopiują całych plików ani baz danych przy każdej zmianie, lecz wykorzystują współdzielenie strukturalne (Structural Sharing), czyli fizyczne używanie tych samych, niezmiennych bloków danych przez wiele wersji jednocześnie.

Drzewo Merkle'a

- Merkle Tree to drzewiasta struktura danych
- Używa haszowania do weryfikacji zawartości dużych zestawów danych
- Każdy węzeł przechowuje hasz swoich dzieci
- Zapewnia integralność danych bez pobierania wszystkiego



Uszkodzenie przy utracie zasilania

Systemy te eliminują ryzyko uszkodzenia danych przy zapisie (Copy-on-Write), ponieważ nigdy nie nadpisują starych informacji, lecz tworzą nowe ścieżki w drzewie Merkle.

Źródła

baeldung.com

codilime.com

[geeksforgeeks.org](https://www.geeksforgeeks.org)

dev.to

opendatastructures.org

brilliant.org

samouczekprogramisty.pl

ocw.mit.edu.pl

en.wikipedia.org

scoutamp.org

wladcysieci.pl

[w3schools.com](https://www.w3schools.com)